

Placement Empowerment Program
Cloud Computing and DevOps Centre

Implement Auto-scaling in the Cloud

Set up an auto-scaling group for your cloud VMs to handle variable workloads.

Name: Neithal Pillai

Department : IT



Auto Scaling is a key feature in AWS that allows instances to scale dynamically based on workload demand. This PoC aims to set up an **Auto Scaling Group (ASG)** with a **Launch Template** and implement a scaling policy based on CPU

usage. The goal is to ensure that instances are automatically launched or terminated as needed to maintain system efficiency and cost-effectiveness.

Objective

- Create a **Launch Template** for virtual machines (VMs) in AWS.
- Set up an **Auto Scaling Group** that dynamically adjusts the number of instances based on CPU usage.
- Define **Scaling Policies** to automatically scale up or down as per workload.
- Simulate high CPU usage to verify auto-scaling functionality.
- Monitor and validate that scaling policies are effectively managing resource allocation.

Importance

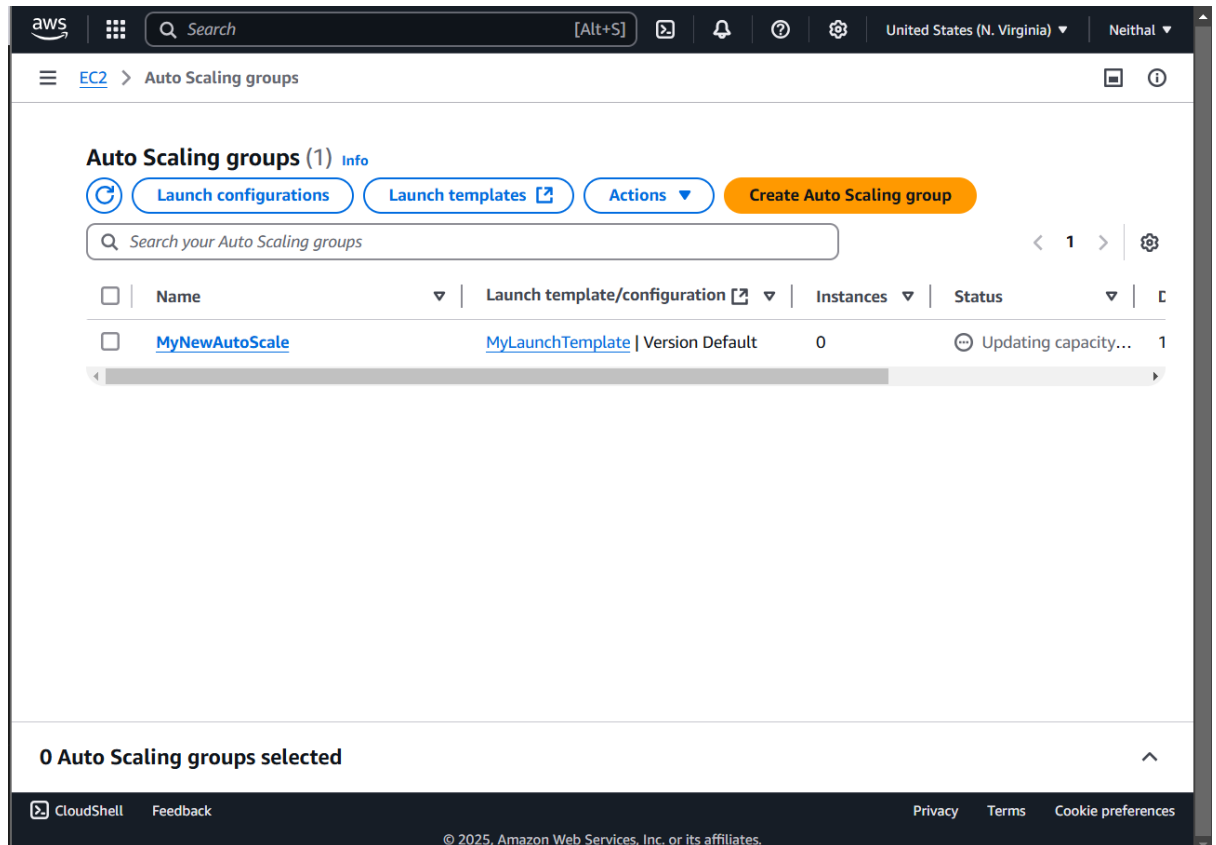
- **Cost Efficiency:** Ensures resources are allocated only when required, reducing costs.
- **High Availability:** Automatically replaces failed instances to maintain uptime.
- **Performance Optimization:** Handles increased traffic without manual intervention.
- **Automated Resource Management:** Enhances system reliability by adjusting resources dynamically.

Steps for Execution

Step 1: Create a Launch Template

1. Open the AWS Management Console and navigate to the EC2 Dashboard.
2. Click on Launch Templates and create a new Launch Template.
3. Provide a name for the template.
4. Select an appropriate Amazon Machine Image (AMI), such as Ubuntu 22.04 or Amazon Linux 2.
5. Choose an instance type based on resource requirements.

6. Configure security groups to allow necessary access.
7. Define any user data scripts needed for instance setup.
8. Save and create the Launch Template.



Step 2: Create an Auto Scaling Group (ASG)

1. Navigate to Auto Scaling Groups and create a new ASG.
2. Select the previously created Launch Template.
3. Provide a name for the Auto Scaling Group.
4. Choose an appropriate Virtual Private Cloud (VPC) and subnets.
5. Define the desired, minimum, and maximum capacity for the ASG.
6. Configure scaling policies to adjust instance count based on CPU utilization.
7. Attach a load balancer (if applicable) to distribute traffic across instances.
8. Review and create the Auto Scaling Group.

The screenshot shows the AWS Management Console interface for creating an Auto Scaling group. The breadcrumb navigation indicates the path: EC2 > Auto Scaling groups > Create Auto Scaling group. On the left, a vertical progress bar shows seven steps: Step 1 (Choose launch template, selected), Step 2 (Choose instance launch options), Step 3 (optional, Integrate with other services), Step 4 (optional, Configure group size and scaling), Step 5 (optional, Add notifications), Step 6 (optional, Add tags), and Step 7 (Review).

The main content area is titled 'Choose launch template' with an 'Info' icon. Below the title, it says 'Specify a launch template that contains settings common to all EC2 instances that are launched by this Auto Scaling group.'

There are two main sections for configuration:

- Name:** Labeled 'Auto Scaling group name', it instructs to 'Enter a name to identify the group.' A text input field contains 'MyNewAutoScale'. A note below states: 'Must be unique to this account in the current Region and no more than 255 characters.'
- Launch template:** Labeled 'Launch template', it instructs to 'Choose a launch template that contains the instance-level settings, such as the Amazon Machine Image (AMI), instance type, key pair, and security groups.' A dropdown menu shows 'MyLaunchTemplate'. To the right of the dropdown is a circular refresh icon. Below the dropdown is a link 'Create a launch template'. Underneath, there is a 'Version' section with a dropdown showing 'Default (1)' and another circular refresh icon. A link 'Create a launch template version' is also present.

A blue information box at the top of the 'Launch template' section contains the following text: 'For accounts created after May 31, 2023, the EC2 console only supports creating Auto Scaling groups with launch templates. Creating Auto Scaling groups with launch configurations is not recommended but still available via the CLI and API until December 31, 2023.'

The footer of the console shows 'CloudShell', 'Feedback', and copyright information: '© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences'.

Step 3: Configure Scaling Policies

1. Open the Auto Scaling Group and navigate to the Scaling Policies section.
2. Choose a Target Tracking Scaling Policy.
3. Set the metric as **Average CPU Utilization**.
4. Define a target utilization percentage (e.g., 50%).
5. Specify cooldown periods to avoid frequent scaling actions.
6. Save the policy.

The screenshot shows the 'User data - optional' section of the AWS console. It includes an 'Info' icon and the instruction: 'Upload a file with your user data or enter it in the field.'

There is a button with an upload icon and the text 'Choose file'.

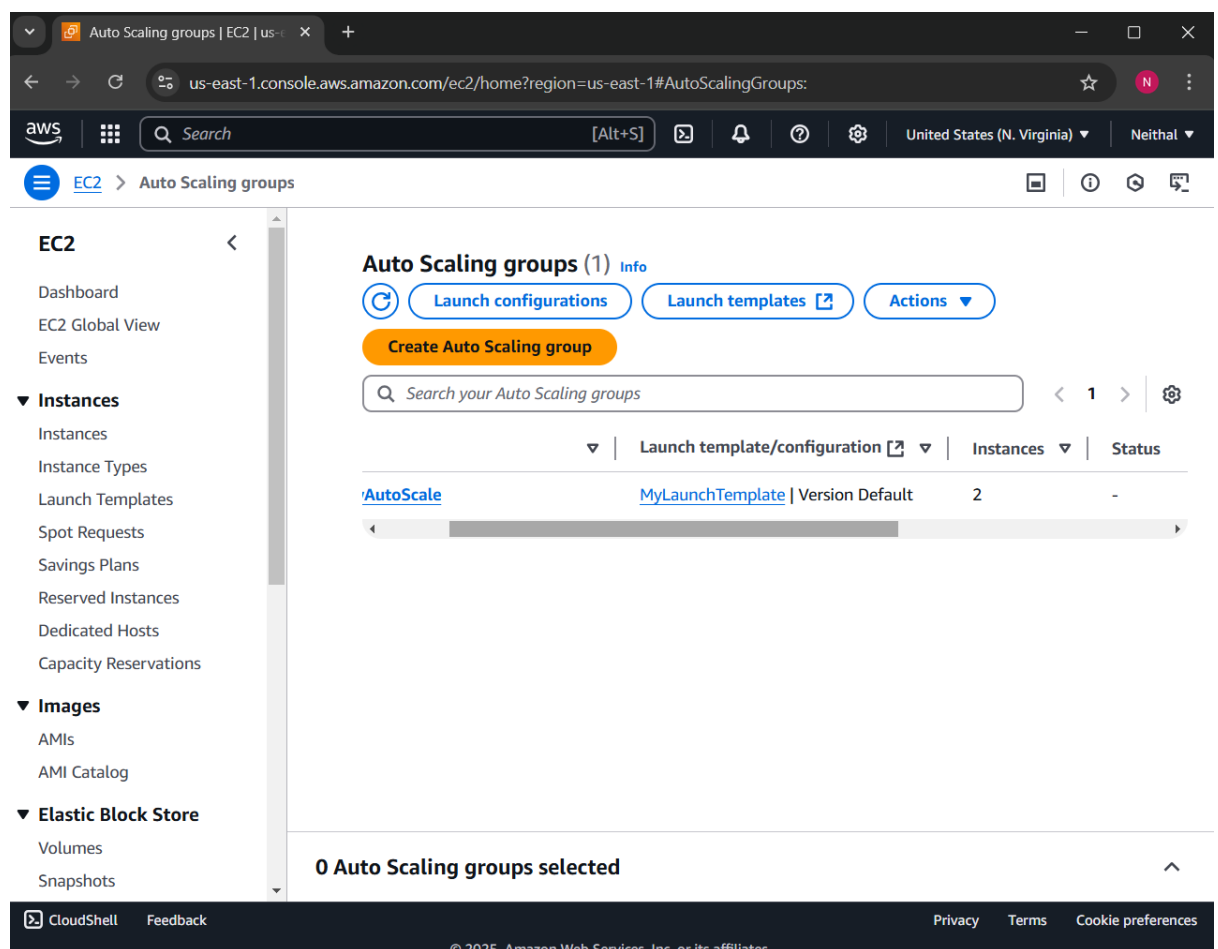
Below this is a large text area containing the following text:

```
#!/bin/bash
yum update -y
yum install -y stress
```

At the bottom of the section, there is a checkbox labeled 'User data has already been base64 encoded'.

Step 4: Test Auto Scaling

1. Identify the instance launched by the Auto Scaling Group in the EC2 Dashboard.
2. Connect to the instance using SSH.
3. Install necessary tools to simulate CPU load.
4. Increase CPU usage to test the scaling mechanism.
5. Monitor the Auto Scaling Group to confirm that additional instances are launched.
6. Check the CloudWatch metrics to verify CPU utilization trends.
7. Observe if new instances are being added automatically as demand increases.



Step 5: Verify Scaling Down

1. Stop the simulated CPU load on all instances.
2. Observe the Auto Scaling Group and check if excess instances are terminated automatically.

3. Review CloudWatch metrics to confirm CPU utilization has decreased.
4. Ensure that the number of active instances returns to the minimum defined in the scaling policy.

Conclusion

The PoC successfully demonstrated AWS Auto Scaling in action. Instances were dynamically launched based on CPU load and terminated when load reduced, ensuring cost efficiency and optimized resource utilization. Additional configurations, such as CloudWatch monitoring and SNS notifications, can further enhance operational efficiency.

What I Learned

- How to create and configure Launch Templates in AWS.
- Setting up Auto Scaling Groups for automatic resource management.
- Implementing Scaling Policies to optimize infrastructure costs.
- Simulating high CPU usage and validating automatic scaling responses.
- Monitoring and verifying AWS Auto Scaling Group performance.
- Enhancing the scalability strategy using CloudWatch, SNS, and CloudFormation.